

완전 초보자를 위한 실습 매뉴얼

OpenClaw로 오라클 VPS 안에 개인 PT 트레이너 고용하기

OpenClaw, Gemini API, Telegram Bot, SQLite, 웹 대시보드로 만드는 개인 AI PT 트레이너 시스템

작성일: 2026-05-23

대상 독자: 개발 경험이 거의 없는 사용자

권장 DB: SQLite 기본, PostgreSQL 확장 선택

목차

OpenClaw로 오라클 VPS 안에 개인 PT 트레이너 고 용하기

최종적으로 만들 시스템
왜 SQLite를 추천하는가
전체 실습 흐름
준비물
이 매뉴얼의 기본 가정
보안과 개인정보 주의사항

1장. Oracle VPS 준비

VPS란 무엇인가
Oracle A1 인스턴스 권장 사양
인스턴스 생성 후 꼭 확인할 것
서버 접속 명령어 예시

2장. 서버 기본 준비

서버 업데이트
필요한 프로그램 설치
프로젝트 폴더 만들기
가상환경 만들기
Python 패키지 설치

3장. Gemini API 키 준비

Gemini API 키란 무엇인가
API 키 만들기
모델 선택

4장. Telegram 봇 준비

Telegram 봇이 하는 일
BotFather로 봇 만들기
내 Telegram Chat ID 확인

5장. 환경 설정 파일 만들기

.env 파일이란 무엇인가
.env 파일 만들기
.env 파일 권한 보호

6장. 데이터베이스 만들기

저장할 데이터

SQLite DB 초기화 파일 만들기

DB 파일 확인

7장. OpenClaw 설치와 역할

OpenClaw가 필요한 이유
OpenClaw 설치 전 확인
OpenClaw 설치
설치가 안 될 때 대체 방법
OpenClaw 설정 파일의 쉬운 이해
OpenClaw 설치 위치

8장. PT 트레이너 지시문 작성

AGENTS.md 작성
SOUL.md 작성
USER.md 작성

9장. 데이터 저장 스크립트 만들기

폴더 구조 만들기
공통 설정 파일 만들기
DB 연결 파일 만들기
간단 기록 스크립트 만들기

10장. Gemini 분석 스크립트 만들기

Gemini 연결 테스트
최근 데이터 조회 파일 만들기
일일 리포트 만들기
주간 리포트 만들기

11장. Telegram 발송 기능 만들기

메시지 보내기 스크립트
일일 리포트를 Telegram으로 보내기
주간 리포트를 Telegram으로 보내기

12장. Telegram 입력 받기

초보자용 입력 방식
Telegram Polling 봇 만들기

13장. 웹 대시보드 만들기

대시보드에서 볼 내용
Flask 앱 만들기

Oracle 방화벽 확인

14장. systemd로 항상 실행하기

왜 systemd가 필요한가

Telegram 봇 서비스 만들기

Dashboard 서비스 만들기

15장. Nginx로 대시보드 연결

Nginx가 하는 일

Nginx 설정 파일 만들기

16장. 자동 리포트 예약

Cron이란 무엇인가

아침 알람 스크립트

Cron 등록

17장. 실제 사용 예시

기본 입력 예시

일일 리포트 요청

주간 리포트 요청

18장. 점검 스크립트

전체 상태 확인 파일 만들기

19장. 자주 나는 오류

SSH 접속이 안 될 때

Telegram 메시지가 안 올 때

Gemini 오류가 날 때

대시보드가 안 열릴 때

DB에 기록이 안 쌓일 때

20장. 운영 루틴

매일 할 일

매주 할 일

좋은 입력 습관

21장. 백업

왜 백업이 필요한가

SQLite 백업

자동 백업 Cron

22장. 보안 강화

기본 보안 체크리스트

대시보드 공개 범위

23장. PostgreSQL로 확장하고 싶을 때

PostgreSQL이 필요한 경우

마이그레이션 개념

24장. PDF 변환 전 정리

PDF에 넣으면 좋은 요소

PDF 변환 전 체크리스트

25장. 전체 완성 체크리스트

부록 A. 명령어 모음

서비스 상태 확인

서비스 재시작

로그 확인

DB 확인

부록 B. PT 트레이너용 프롬프트 예시

일일 피드백 프롬프트

주간 피드백 프롬프트

부록 C. 참고 자료

마무리

OpenClaw로 오라클 VPS 안에 개인 PT 트레이너 고용하기

완전 초보자를 위한 실습 매뉴얼

작성일: 2026-05-23 대상 독자: 개발 경험이 거의 없는 사용자 목표 결과물: Telegram으로 대화하고, OpenClaw와 Gemini API가 운동/식단을 분석하며, Oracle Cloud VPS 안에 기록을 저장하고, 웹 대시보드로 확인할 수 있는 개인 AI PT 트레이너 시스템

문서 사용 방법

이 문서는 코딩을 공부하기 위한 문서가 아니라, 그대로 따라 해서 나만의 AI PT 트레이너 시스템을 만드는 실습 매뉴얼입니다.

초보자를 기준으로 작성했기 때문에 명령어의 의미를 전부 이해하지 못해도 괜찮습니다. 다만 다음 원칙은 꼭 지켜야 합니다.

- 명령어는 한 줄씩 복사해서 실행합니다.
- 오류가 나오면 다음 단계로 넘어가지 않습니다.
- API 키, 봇 토큰, 비밀번호는 절대 공개된 곳에 올리지 않습니다.
- 건강, 질병, 통증, 약물 관련 판단은 AI에게만 맡기지 않습니다.
- 운동 중 통증, 어지럼증, 호흡 곤란이 있으면 운동을 중단하고 전문가에게 상담합니다.

이 문서는 PDF 변환을 전제로 작성되어 있습니다. 각 장은 PDF에서 독립적인 페이지 묶음처럼 읽히도록 구성했습니다.

최종적으로 만들 시스템

우리가 만들 시스템은 다음과 같이 동작합니다.

1. 사용자가 Telegram에 운동, 식단, 체중, 수면, 컨디션을 입력합니다.
2. Telegram 봇이 그 메시지를 OpenClaw에게 전달합니다.
3. OpenClaw는 메시지 내용을 읽고 기록할 항목을 구분합니다.
4. Gemini API는 기록을 바탕으로 PT 트레이너처럼 분석합니다.
5. SQLite 데이터베이스에 기록이 저장됩니다.
6. 매일 저녁 일일 리포트가 생성됩니다.
7. 매주 일요일 주간 리포트가 생성됩니다.
8. 웹 대시보드에서 체중, 운동, 식단, 리포트를 확인합니다.

비유하면 다음과 같습니다.

구성 요소	쉬운 비유	실제 역할
Oracle VPS	24시간 켜져 있는 사무실	AI PT 시스템이 계속 실행되는 서버
OpenClaw	고용한 AI 직원	메시지를 받고 도구를 실행하는 에이전트
Gemini API	AI 직원의 두뇌	분석, 요약, 피드백 생성
Telegram Bot	PT와 대화하는 창구	사용자의 입력을 받는 메신저
SQLite DB	PT의 기록 노트	운동/식단/체중 데이터 저장
Dashboard	PT 기록판	기록과 리포트를 웹으로 확인

왜 SQLite를 추천하는가

이 문서에서는 기본 데이터베이스로 SQLite를 사용합니다.
완전 초보자에게는 PostgreSQL보다 SQLite가 적합합니다.

항목	SQLite	PostgreSQL
설치 난이도	매우 쉬움	중간 이상
계정/비밀번호 설정	거의 없음	필요함
백업	파일 하나 복사	명령어 필요
개인용 PT 기록	충분함	충분함
여러 사용자 동시 사용	약함	강함
장기 서비스 운영	보통	좋음
초보자 추천도	높음	낮음

이 프로젝트의 1차 목표는 “내가 혼자 쓰는 AI PT 트레이너”입니다. 하루에 몇 번 기록을 저장하고, 매일/매주 리포트를 만드는 수준에서는 SQLite로 충분합니다.

PostgreSQL은 다음 상황에서 고려하면 됩니다.

- 여러 명이 동시에 쓰는 서비스로 키우고 싶을 때
- 운동 기록이 수십만 건 이상 쌓일 때
- 외부 서비스와 복잡하게 연동할 때
- 데이터 권한, 계정, 백업 정책을 더 엄격하게 관리해야 할 때

따라서 이 매뉴얼은 SQLite로 완성한 뒤, 부록에서 PostgreSQL로 확장하는 방향을 제안합니다.

전체 실습 흐름

이 매뉴얼은 다음 순서로 진행합니다.

- 1. 준비물 확인
- 2. Oracle Cloud VPS 준비
- 3. VPS에 접속하기
- 4. 기본 프로그램 설치
- 5. 프로젝트 폴더 만들기
- 6. Gemini API 키 준비
- 7. Telegram 봇 만들기
- 8. SQLite 데이터베이스 만들기
- 9. OpenClaw 설치 및 기본 설정
- 10. PT 트레이너 지시문 작성
- 11. Telegram 메시지 기록 기능 만들기
- 12. 일일 리포트 만들기
- 13. 주간 리포트 만들기
- 14. 웹 대시보드 만들기
- 15. 자동 실행 설정
- 16. 테스트
- 17. 운영 방법
- 18. 오류 해결
- 19. 보안 점검
- 20. PDF 변환 전 체크리스트

준비물

실습 전에 다음 항목이 필요합니다.

준비물	필요 여부	설명
Oracle Cloud 계정	필수	무료 VPS를 만들기 위해 필요
Oracle A1 인스턴스	필수	OpenClaw를 24시간 실행할 서버
Telegram 계정	필수	PT 트레이너와 대화할 메신저
Telegram Bot Token	필수	봇 연결에 필요
Gemini API Key	필수	AI 분석에 필요
SSH 접속 도구	필수	서버에 접속하기 위해 필요
도메인	선택	대시보드를 예쁜 주소로 접속하고 싶을 때
기초 운동 목표	권장	감량, 증량, 체력 개선 등

Windows 사용자는 PowerShell 또는 Windows Terminal을 사용하면 됩니다.

Mac 사용자는 기본 Terminal을 사용하면 됩니다.

이 매뉴얼의 기본 가정

실습 환경은 다음 기준으로 설명합니다.

- 서버: Ubuntu 22.04 또는 Ubuntu 24.04
- 서버 위치: Oracle Cloud A1 Always Free 인스턴스
- 사용자 계정: `ubuntu`
- 프로젝트 경로: `~/pt_system`
- 데이터베이스: SQLite
- AI 모델: Gemini API
- 기본 모델명: `gemini-2.0-flash`
- 메신저: Telegram Bot
- 대시보드: Flask 웹 앱
- 웹 서버 포트: `5000`

Gemini API 공식 문서 기준으로 `gemini-2.0-flash` 는 텍스트, 이미지, 오디오, 비디오 입력을 받을 수 있고 텍스트를 출력하는 모델입니다. 이 매뉴얼에서는 운동/식단 텍스트 분석을 중심으로 사용합니다.

보안과 개인정보 주의사항

이 시스템에는 개인 건강 데이터가 들어갑니다.

다음 정보는 민감 정보로 취급해야 합니다.

- 체중
- 체지방률
- 식단
- 수면 시간
- 운동 수행 능력
- 부상 이력
- 질병 이력
- 복용 중인 약
- 신체 사진

이 매뉴얼에서는 기본적으로 개인용 시스템을 가정합니다. 다른 사람에게 계정을 공유하거나, 공개 웹사이트처럼 운영하지 마세요.

특히 다음 파일은 외부에 공개하면 안 됩니다.

- `.env`
- `pt_data.db`
- SSH 개인키
- Telegram Bot Token
- Gemini API Key

1장. Oracle VPS 준비

VPS란 무엇인가

VPS는 인터넷에 항상 켜져 있는 내 컴퓨터라고 생각하면 됩니다.

내 노트북은 꺼지면 프로그램도 멈춥니다. 하지만 VPS는 24시간 켜져 있으므로 Telegram 메시지를 계속 받을 수 있습니다.

이 프로젝트에서 VPS는 다음 역할을 합니다.

- OpenClaw 실행
- Telegram 봇 실행
- 데이터베이스 저장
- 매일/매주 리포트 자동 생성
- 웹 대시보드 제공

Oracle A1 인스턴스 권장 사양

개인용 AI PT 시스템은 아주 큰 서버가 필요하지 않습니다.

권장 사양은 다음과 같습니다.

항목	권장값
CPU	1 OCPU 이상
메모리	6GB 이상 권장
디스크	50GB 이상
OS	Ubuntu 22.04 또는 24.04
아키텍처	ARM64 가능

OpenClaw, Python, Flask, SQLite 정도는 A1 무료 인스턴스에서도 충분히 운영 가능합니다.

인스턴스 생성 후 꼭 확인할 것

Oracle Cloud에서 인스턴스를 만든 뒤 다음을 확인합니다.

- 공인 IP가 있는가
- SSH 키를 저장했는가
- Ubuntu로 생성했는가
- 보안 목록에서 필요한 포트를 열 수 있는가
- 인스턴스가 Running 상태인가

초보자가 가장 많이 실수하는 부분은 SSH 키를 잃어버리는 것입니다.

SSH 키는 서버에 들어가는 열쇠입니다. 잃어버리면 새로 접속하기 어렵습니다. 반드시 안전한 폴더에 보관하세요.

서버 접속 명령어 예시

Windows PowerShell 또는 Mac Terminal에서 다음 형식으로 접속합니다.

```
ssh -i "내_키_파일_경로" ubuntu@서버_공인_IP
```

예시:

```
ssh -i "C:\Users\me\Downloads\openclaw.pem" ubuntu@123.123.123.123
```

접속에 성공하면 화면 왼쪽에 다음과 비슷한 표시가 나옵니다.

```
ubuntu@instance-name:~$
```

이 표시가 보이면 서버 안에 들어온 것입니다.

2장. 서버 기본 준비

서버 업데이트

서버에 처음 접속하면 먼저 기본 업데이트를 합니다.

```
sudo apt update
```

```
sudo apt upgrade -y
```

업데이트는 시간이 조금 걸릴 수 있습니다.

필요한 프로그램 설치

다음 명령어로 기본 도구를 설치합니다.

```
sudo apt install -y python3 python3-pip python3-venv git curl unzip sqlite3 nginx
```

설치가 끝나면 Python 버전을 확인합니다.

```
python3 --version
```

Python 3.9 이상이면 충분합니다.

프로젝트 폴더 만들기

이 프로젝트의 모든 파일은 `~/pt_system` 폴더에 넣겠습니다.

```
mkdir -p ~/pt_system
```

```
cd ~/pt_system
```

앞으로 대부분의 작업은 이 폴더 안에서 진행합니다.

가상환경 만들기

Python 프로젝트는 가상환경을 만들어 사용하는 것이 좋습니다.

```
python3 -m venv venv
```

```
source venv/bin/activate
```

성공하면 명령줄 앞에 (venv) 가 붙습니다.

예시:

```
(venv) ubuntu@instance:~/pt_system$
```

Python 패키지 설치

다음 패키지를 설치합니다.

```
pip install --upgrade pip
```

```
pip install python-dotenv requests flask google-genai gunicorn
```

각 패키지의 역할은 다음과 같습니다.

패키지	역할
python-dotenv	.env 파일에서 API 키 읽기
requests	Telegram API 호출
flask	웹 대시보드 만들기
google-genai	Gemini API 사용
gunicorn	Flask 앱 운영용 실행

3장. Gemini API 키 준비

Gemini API 키란 무엇인가

Gemini API 키는 Google의 AI 모델을 사용할 수 있게 해주는 열쇠입니다.

이 프로젝트에서는 Gemini가 다음 일을 합니다.

- 운동 기록 요약
 - 식단 기록 분석
 - 체중 변화 해석
 - 일일 리포트 작성
 - 주간 리포트 작성
 - 다음 주 운동/식단 조언
-

API 키 만들기

1. Google AI Studio에 접속합니다.
2. API Key 메뉴로 이동합니다.
3. 새 API 키를 생성합니다.
4. 생성된 키를 복사합니다.
5. 이 키를 다른 사람에게 보여주지 않습니다.

API 키는 비밀번호와 같습니다. 유출되면 다른 사람이 내 API 사용량을 써버릴 수 있습니다.

모델 선택

이 매뉴얼에서는 기본 모델로 다음을 사용합니다.

```
gemini-2.0-flash
```

선택 이유는 다음과 같습니다.

- 빠릅니다.
- 비용 부담이 낮습니다.
- 텍스트 분석에 충분합니다.
- 운동/식단 기록처럼 짧은 입력을 처리하기 좋습니다.

더 높은 품질이 필요하면 나중에 Gemini 2.5 계열 모델을 검토할 수 있습니다. 다만 초보자 실습에서는 모델을 자주 바꾸지 않는 것이 좋습니다.

4장. Telegram 봇 준비

Telegram 봇이 하는 일

Telegram 봇은 AI PT 트레이너와 대화하는 입구입니다.

사용자는 Telegram에 다음처럼 입력합니다.

오늘 운동 :
스쿼트 60kg 5회 5세트
벤치프레스 40kg 8회 4세트

식단 :
아침 삶은 계란 2개, 바나나 1개
점심 닭가슴살 200g, 밥 반 공기

체중 78.2kg, 수면 6.5시간, 컨디션 보통

봇은 이 메시지를 서버로 보내고, 서버는 기록을 저장한 뒤 답장을 보냅니다.

BotFather로 봇 만들기

Telegram에서 BotFather 를 검색합니다.

다음 순서로 진행합니다.

1. BotFather와 대화를 시작합니다.
2. /newbot 입력
3. 봇 이름 입력
4. 봇 username 입력
5. Bot Token 복사

Bot Token은 다음처럼 생겼습니다.

1234567890:AAExampleExampleExampleExample

이 토큰은 절대 공개하면 안 됩니다.

내 Telegram Chat ID 확인

봇이 나에게 메시지를 보내려면 내 Chat ID가 필요합니다.

가장 쉬운 방법은 나중에 테스트 스크립트에서 자동으로 확인하는 것입니다.

먼저 만든 봇에게 아무 메시지도 보냅니다.

예:

안녕

그 다음 서버에서 Telegram 업데이트를 조회합니다.

```
curl "https://api.telegram.org/bot봇_토큰/getUpdates"
```

결과 안에서 `"chat":{"id":숫자}` 부분을 찾습니다.

그 숫자가 내 Chat ID입니다.

5장. 환경 설정 파일 만들기

.env 파일이란 무엇인가

.env 파일은 비밀 정보를 따로 보관하는 파일입니다.

여기에는 다음을 넣습니다.

- Gemini API Key
- Telegram Bot Token
- Telegram Chat ID
- 사용할 Gemini 모델명
- 데이터베이스 파일 경로

.env 파일 만들기

프로젝트 폴더에서 다음 명령어를 실행합니다.

```
nano .env
```

아래 내용을 붙여넣습니다.

```
GEMINI_API_KEY=여기에 _Gemini_API_키  
GEMINI_MODEL=gemini-2.0-flash  
TELEGRAM_BOT_TOKEN=여기에 _Telegram_봇_토큰  
TELEGRAM_CHAT_ID=여기에 _내_Chat_ID  
DATABASE_PATH=/home/ubuntu/pt_system/pt_data.db
```

저장 방법:

1. `Ctrl + X`
2. `Y`
3. `Enter`

.env 파일 권한 보호

비밀 정보가 들어 있으므로 권한을 제한합니다.

```
chmod 600 .env
```

권한을 확인합니다.

```
ls -l .env
```

앞부분이 다음처럼 보이면 됩니다.

-rw-----

6장. 데이터베이스 만들기

저장할 데이터

AI PT 시스템은 다음 데이터를 저장합니다.

데이터	예시
운동	스쿼트 60kg 5회 5세트
식단	닭가슴살 200g, 밥 반 공기
체중	78.2kg
수면	6.5시간
컨디션	좋음, 보통, 피곤함
리포트	일일 분석, 주간 분석

SQLite DB 초기화 파일 만들기

다음 파일을 만듭니다.

```
nano init_db.py
```

아래 코드를 붙여넣습니다.

```

import sqlite3
from pathlib import Path

DB_PATH = Path("/home/ubuntu/pt_system/pt_data.db")

conn = sqlite3.connect(DB_PATH)
cur = conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS workouts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    record_date TEXT NOT NULL,
    exercise_name TEXT NOT NULL,
    weight_kg REAL,
    reps INTEGER,
    sets INTEGER,
    memo TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS meals (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    record_date TEXT NOT NULL,
    meal_type TEXT,
    food_text TEXT NOT NULL,
    calories_estimate INTEGER,
    protein_g REAL,
    memo TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS vitals (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    record_date TEXT NOT NULL,
    body_weight_kg REAL,
    sleep_hours REAL,
    condition_text TEXT,
    memo TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS reports (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    report_type TEXT NOT NULL,
    start_date TEXT NOT NULL,
    end_date TEXT NOT NULL,
    report_text TEXT NOT NULL,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
)
""")

```

```
cur.execute("""
CREATE TABLE IF NOT EXISTS raw_messages (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    source TEXT NOT NULL,
    message_text TEXT NOT NULL,
    processed INTEGER DEFAULT 0,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
)
""")

conn.commit()
conn.close()

print(f"Database initialized: {DB_PATH}")
```

실행합니다.

```
python3 init_db.py
```

성공하면 다음과 비슷하게 나옵니다.

```
Database initialized: /home/ubuntu/pt_system/pt_data.db
```

DB 파일 확인

```
ls -l pt_data.db
```

DB 안의 테이블을 확인합니다.

```
sqlite3 pt_data.db ".tables"
```

다음 테이블이 보이면 성공입니다.

```
meals raw_messages reports vitals workouts
```

7장. OpenClaw 설치와 역할

OpenClaw가 필요한 이유

단순히 Gemini API만 쓰면 한 번 질문하고 한 번 답을 받는 구조가 됩니다.

하지만 OpenClaw를 사용하면 다음이 가능해집니다.

- Telegram 메시지를 계속 받아 처리
- 사용자의 장기 목표 기억
- 정해진 규칙에 따라 도구 실행
- 매일/매주 자동 작업 수행
- 여러 파일과 기록을 참고해서 답변

이 프로젝트에서 OpenClaw는 “AI PT 트레이너 직원” 역할을 합니다.

OpenClaw 설치 전 확인

OpenClaw 공식 설치 문서 기준으로 Node.js 24가 권장되며, Node.js 22.14 이상도 지원됩니다. 초보자는 설치 스크립트를 사용하는 것이 가장 쉽습니다.

먼저 Node.js가 설치되어 있는지 확인합니다.

```
node --version
```

Node.js가 없거나 버전이 너무 낮아도 괜찮습니다. 아래 설치 스크립트가 필요한 구성 요소를 함께 설치합니다.

OpenClaw 설치

VPS에서 다음 명령어를 실행합니다.

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

설치 과정에서 온보딩 화면이 나오면 사용하는 모델 제공자와 API 키를 입력합니다.

이 매뉴얼에서는 OpenClaw의 기본 대화 모델 설정과 별개로, PT 리포트 생성 스크립트에서 Gemini API를 직접 호출합니다. 따라서 OpenClaw 온보딩에서 어떤 모델을 고르더라도, `.env` 파일의 `GEMINI_API_KEY` 와 `GEMINI_MODEL` 은 반드시 따로 설정해야 합니다.

설치가 끝나면 새 터미널을 열거나 다음 명령어로 현재 셸 설정을 다시 읽습니다.

```
source ~/.bashrc
```

OpenClaw 설치 확인:

```
openclaw --version
```

상태 점검:

```
openclaw doctor
```

Gateway 상태 확인:

```
openclaw gateway status
```

정상이라면 OpenClaw 명령어가 인식되고, Gateway가 실행 중이라는 메시지를 볼 수 있습니다.

설치가 안 될 때 대체 방법

설치 스크립트가 실패하면 npm 방식으로 설치할 수 있습니다.

```
npm install -g openclaw@latest
```

그 다음 온보딩을 실행합니다.

```
openclaw onboard --install-daemon
```

openclaw: command not found 가 나오면 전역 npm 실행 경로가 PATH에 없을 수 있습니다.

다음 명령어로 확인합니다.

```
npm prefix -g
```

출력된 경로 뒤에 /bin 을 붙인 값이 PATH에 포함되어야 합니다.

OpenClaw 설정 파일의 쉬운 이해

OpenClaw는 여러 Markdown 파일을 읽어서 AI의 행동을 정합니다.

파일	쉬운 설명	이 프로젝트에서의 역할
AGENTS.md	업무 지시서	PT 트레이너가 해야 할 일
SOUL.md	성격 설명서	말투, 태도, 금지 행동
USER.md	회원 프로필	사용자의 목표, 제한사항
MEMORY.md	장기 기억	누적된 중요한 정보
TOOLS.md	도구 설명서	사용할 수 있는 스크립트

OpenClaw 공식 문서에서도 SOUL.md 는 에이전트의 성격, 경계, 톤을 정의하는 파일로 설명됩니다. 이 매뉴얼에서는 PT 트레이너 역할에 맞게 AGENTS.md 와 SOUL.md 를 작성합니다.

OpenClaw 설치 위치

OpenClaw 설치 방식은 사용하는 배포판이나 버전에 따라 달라질 수 있습니다.

이 매뉴얼에서는 다음 원칙으로 설명합니다.

- OpenClaw는 VPS에서 실행합니다.
- OpenClaw의 작업 폴더는 `~/.openclaw/workspace` 를 사용합니다.
- PT 시스템 파일은 `~/pt_system` 에 둡니다.
- OpenClaw는 필요할 때 `~/pt_system` 의 Python 스크립트를 실행합니다.

OpenClaw 설치 후 다음 폴더가 있는지 확인합니다.

```
ls ~/.openclaw
```

작업 폴더가 없다면 만듭니다.

```
mkdir -p ~/.openclaw/workspace
```

8장. PT 트레이너 지시문 작성

AGENTS.md 작성

다음 파일을 엽니다.

```
nano ~/.openclaw/workspace/AGENTS.md
```

아래 내용을 붙여넣습니다.

개인 AI PT 트레이너 에이전트 지시문

기본 역할

당신은 사용자의 개인 AI PT 트레이너이자 식단 기록 코치입니다.
사용자의 Telegram 메시지를 바탕으로 운동, 식단, 체중, 수면, 컨디션을 정리하고 저장합니다.
사용자의 목표 달성을 돕기 위해 매일 짧은 피드백과 매주 깊이 있는 리포트를 제공합니다.

핵심 목표

1. 사용자가 쉽게 기록을 남기도록 돕는다.
2. 운동 기록을 누적해서 진행 상황을 보여준다.
3. 식단 기록을 바탕으로 과식, 단백질 부족, 수분 부족 등을 부드럽게 알려준다.
4. 체중과 수면 변화를 함께 보고 무리하지 않게 조언한다.
5. 사용자가 꾸준히 실천할 수 있도록 현실적인 피드백을 준다.

절대 하지 말아야 할 일

1. 의학적 진단을 내리지 않는다.
2. 질병 치료법을 단정하지 않는다.
3. 통증이 있는데도 운동을 강행하라고 말하지 않는다.
4. 극단적인 저칼로리 식단을 권하지 않는다.
5. 사용자의 API 키, 토큰, 비밀번호를 메시지로 출력하지 않는다.
6. 사용자의 허락 없이 시스템 파일을 삭제하지 않는다.

기록 처리 규칙

사용자가 운동, 식단, 체중, 수면, 컨디션을 입력하면 pt_system의 기록 스크립트를 사용한다.
기록이 완료되면 다음 형식으로 짧게 답한다.

[기록 완료]

- 운동: N개
- 식단: N개
- 바이탈: 저장됨 또는 없음
- 오늘의 한마디: 한두 문장

리포트 규칙

일일 리포트는 짧고 실천 중심으로 작성한다.
주간 리포트는 추세, 잘한 점, 개선점, 다음 주 계획을 포함한다.
리포트는 무조건 한국어로 작성한다.
사용자를 비난하지 않고, 구체적인 다음 행동을 제안한다.

말투

친근하지만 전문적인 PT 트레이너처럼 말한다.
과장하지 않는다.
데이터가 부족하면 부족하다고 말한다.
추측은 추측이라고 밝힌다.

저장합니다.

SOUL.md 작성

다음 파일을 엽니다.

```
nano ~/.openclaw/workspace/SOUL.md
```

아래 내용을 붙여넣습니다.

```
# PT 트레이너 페르소나

## 정체성

이름: AI PT 코치
역할: 개인 운동 기록 코치, 식단 기록 코치, 생활 습관 피드백 코치

## 성격

- 차분하고 현실적이다.
- 사용자의 실천 가능성을 가장 중요하게 본다.
- 완벽한 하루보다 지속 가능한 루틴을 우선한다.
- 잘한 점을 먼저 짚고, 개선점은 구체적으로 제안한다.
- 운동 초보자의 불안과 귀찮음을 이해한다.

## 말투

- 한국어로 답한다.
- 짧고 명확하게 말한다.
- 기록 답변은 5줄 이내를 기본으로 한다.
- 리포트는 표와 목록을 적절히 사용한다.

## 안전 기준

- 통증, 부상, 질병, 약물 이야기가 나오면 전문가 상담을 권한다.
- 무리한 강량을 권하지 않는다.
- 수면 부족과 과훈련 신호를 가볍게 넘기지 않는다.
- 신체 사진이나 민감 정보를 외부 공유하지 않는다.
```

USER.md 작성

사용자 프로필 파일을 만듭니다.

```
nano ~/.openclaw/workspace/USER.md
```

아래는 예시입니다. 본인 상황에 맞게 바꿉니다.

사용자 프로필

기본 정보

- 이름 또는 별명: 사용자
- 운동 목적: 체지방 감량과 근력 향상
- 운동 수준: 초보자
- 주당 운동 가능 횟수: 주 3회
- 운동 가능 장소: 헬스장
- 선호 운동: 웨이트 트레이닝, 걷기
- 피하고 싶은 운동: 무리한 유산소

건강 관련 주의사항

- 통증 부위: 없음
- 질병 이력: 직접 입력
- 복용 약: 직접 입력
- 주의할 점: 통증이 있으면 운동 강도 낮추기

식단 성향

- 선호 음식: 직접 입력
- 싫어하는 음식: 직접 입력
- 알레르기: 직접 입력
- 식사 패턴: 아침 간단히, 점심 일반식, 저녁 조절

코칭 목표

AI PT 코치는 매일 기록을 쉽게 남기도록 돕고, 주간 단위로 운동과 식단을 현실적으로 개선해준다.

9장. 데이터 저장 스크립트 만들기

폴더 구조 만들기

```
mkdir -p ~/pt_system/scripts
```

```
mkdir -p ~/pt_system/reports
```

```
mkdir -p ~/pt_system/dashboard
```

```
mkdir -p ~/pt_system/logs
```

공통 설정 파일 만들기

```
nano ~/pt_system/scripts/config.py
```

```
import os
from pathlib import Path
from dotenv import load_dotenv

BASE_DIR = Path("/home/ubuntu/pt_system")
load_dotenv(BASE_DIR / ".env")

DATABASE_PATH = os.getenv("DATABASE_PATH", str(BASE_DIR / "pt_data.db"))
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY", "")
GEMINI_MODEL = os.getenv("GEMINI_MODEL", "gemini-2.0-flash")
TELEGRAM_BOT_TOKEN = os.getenv("TELEGRAM_BOT_TOKEN", "")
TELEGRAM_CHAT_ID = os.getenv("TELEGRAM_CHAT_ID", "")
```

DB 연결 파일 만들기

```
nano ~/pt_system/scripts/db.py
```

```
import sqlite3
from .config import DATABASE_PATH

def get_conn():
    conn = sqlite3.connect(DATABASE_PATH)
    conn.row_factory = sqlite3.Row
    return conn
```

Python에서 폴더를 패키지처럼 인식하도록 파일을 만듭니다.

```
touch ~/pt_system/scripts/__init__.py
```

간단 기록 스크립트 만들기

초보자용 1차 버전은 완벽한 자연어 파서가 아닙니다. 대신 사용자가 입력한 원문을 저장하고, 필요한 핵심 수치만 간단히 추출합니다.

```
nano ~/pt_system/scripts/save_message.py
```

```

import re
import sys
from datetime import date
from pathlib import Path

sys.path.append(str(Path("/home/ubuntu/pt_system")))

from scripts.db import get_conn

def find_weight(text):
    match = re.search(r"체중\s*([0-9]+(?:\.[0-9]+)?)\s*kg", text, re.IGNORECASE)
    return float(match.group(1)) if match else None

def find_sleep(text):
    match = re.search(r"수면\s*([0-9]+(?:\.[0-9]+)?)\s*시간", text)
    return float(match.group(1)) if match else None

def save_raw_message(text):
    conn = get_conn()
    cur = conn.cursor()
    cur.execute(
        "INSERT INTO raw_messages (source, message_text, processed) VALUES (?, ?, ?)",
        ("telegram", text, 1),
    )
    conn.commit()
    conn.close()

def save_vitals_if_present(text):
    body_weight = find_weight(text)
    sleep_hours = find_sleep(text)

    if body_weight is None and sleep_hours is None and "컨디션" not in text:
        return False

    condition = None
    if "컨디션" in text:
        condition = text.split("컨디션", 1)[-1].strip().splitlines()[0][:100]

    conn = get_conn()
    cur = conn.cursor()
    cur.execute(
        """
        INSERT INTO vitals (record_date, body_weight_kg, sleep_hours, condition_text, memo)
        VALUES (?, ?, ?, ?, ?)
        """,
        (date.today().isoformat(), body_weight, sleep_hours, condition, text),
    )
    conn.commit()
    conn.close()
    return True

def save_workout_if_present(text):
    keywords = ["운동", "스쿼트", "벤치", "데드", "런닝", "걷기", "레그", "랫풀", "로우", "프레스"]
    if not any(k in text for k in keywords):
        return 0

    lines = [line.strip() for line in text.splitlines() if line.strip()]

```

```

count = 0
conn = get_conn()
cur = conn.cursor()

for line in lines:
    if any(k in line for k in keywords):
        cur.execute(
            """
            INSERT INTO workouts (record_date, exercise_name, memo)
            VALUES (?, ?, ?)
            """,
            (date.today().isoformat(), line[:80], line),
        )
        count += 1

conn.commit()
conn.close()
return count

def save_meal_if_present(text):
    keywords = ["식단", "아침", "점심", "저녁", "간식", "닭가슴살", "밥", "고구마", "샐러드", "계란"]
    if not any(k in text for k in keywords):
        return 0

    lines = [line.strip() for line in text.splitlines() if line.strip()]
    count = 0
    conn = get_conn()
    cur = conn.cursor()

    for line in lines:
        if any(k in line for k in keywords):
            meal_type = None
            for candidate in ["아침", "점심", "저녁", "간식"]:
                if candidate in line:
                    meal_type = candidate
                    break
            cur.execute(
                """
                INSERT INTO meals (record_date, meal_type, food_text, memo)
                VALUES (?, ?, ?, ?)
                """,
                (date.today().isoformat(), meal_type, line, line),
            )
            count += 1

    conn.commit()
    conn.close()
    return count

def main():
    text = " ".join(sys.argv[1:]).strip()
    if not text:
        text = sys.stdin.read().strip()

    if not text:
        print("입력된 메시지가 없습니다.")
        return

```



```
save_raw_message(text)
workout_count = save_workout_if_present(text)
meal_count = save_meal_if_present(text)
vitals_saved = save_vitals_if_present(text)

print("[기록 완료]")
print(f"- 운동: {workout_count}개")
print(f"- 식단: {meal_count}개")
print(f"- 바이탈: {'저장됨' if vitals_saved else '없음'}")
print("- 오늘의 한마디: 기록을 남긴 것 자체가 가장 중요한 시작입니다.")

if __name__ == "__main__":
    main()
```

테스트합니다.

```
python3 ~/pt_system/scripts/save_message.py "오늘 운동 스쿼트 60kg 5x5, 점심 닭가슴살 200g, 체중 78.2kg, 수면 6.5시간"
```

10장. Gemini 분석 스크립트 만들기

Gemini 연결 테스트

```
nano ~/pt_system/scripts/test_gemini.py
```

```
from google import genai
from scripts.config import GEMINI_MODEL

client = genai.Client()

response = client.models.generate_content(
    model=GEMINI_MODEL,
    contents="한국어로 한 문장만 답하세요. AI PT 시스템 테스트입니다."
)

print(response.text)
```

실행합니다.

```
cd ~/pt_system
```

```
source venv/bin/activate
```

```
python3 scripts/test_gemini.py
```

정상이라면 한국어 문장이 출력됩니다.

최근 데이터 조회 파일 만들기

```
nano ~/pt_system/reports/fetch_data.py
```

```

from datetime import date, timedelta
from pathlib import Path
import sys

sys.path.append(str(Path("/home/ubuntu/pt_system")))

from scripts.db import get_conn

def fetch_range(days=1):
    end = date.today()
    start = end - timedelta(days=days - 1)

    conn = get_conn()
    cur = conn.cursor()

    cur.execute(
        "SELECT * FROM workouts WHERE record_date BETWEEN ? AND ? ORDER BY record_date, id",
        (start.isoformat(), end.isoformat()),
    )
    workouts = cur.fetchall()

    cur.execute(
        "SELECT * FROM meals WHERE record_date BETWEEN ? AND ? ORDER BY record_date, id",
        (start.isoformat(), end.isoformat()),
    )
    meals = cur.fetchall()

    cur.execute(
        "SELECT * FROM vitals WHERE record_date BETWEEN ? AND ? ORDER BY record_date, id",
        (start.isoformat(), end.isoformat()),
    )
    vitals = cur.fetchall()

    conn.close()

    return {
        "start": start.isoformat(),
        "end": end.isoformat(),
        "workouts": [dict(row) for row in workouts],
        "meals": [dict(row) for row in meals],
        "vitals": [dict(row) for row in vitals],
    }

```

일일 리포트 만들기

```
nano ~/pt_system/reports/daily_report.py
```

```

from pathlib import Path
import sys

sys.path.append(str(Path("/home/ubuntu/pt_system")))

from google import genai
from scripts.config import GEMINI_MODEL
from scripts.db import get_conn
from reports.fetch_data import fetch_range

def build_prompt(data):
    return f"""
당신은 개인 PT 트레이너입니다.
아래의 하루 기록을 보고 한국어로 일일 리포트를 작성하세요.

규칙:
- 의학적 진단 금지
- 잘한 점 2개
- 개선할 점 2개
- 내일 할 행동 3개
- 전체 길이 800자 이내
- 초보자가 이해하기 쉽게 작성

데이터:
{data}
"""

def save_report(start, end, text):
    conn = get_conn()
    cur = conn.cursor()
    cur.execute(
        """
        INSERT INTO reports (report_type, start_date, end_date, report_text)
        VALUES (?, ?, ?, ?)
        """,
        ("daily", start, end, text),
    )
    conn.commit()
    conn.close()

def main():
    data = fetch_range(days=1)
    client = genai.Client()
    response = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=build_prompt(data),
    )
    report = response.text.strip()
    save_report(data["start"], data["end"], report)
    print(report)

if __name__ == "__main__":
    main()

```

실행합니다.

```
python3 ~/pt_system/reports/daily_report.py
```

주간 리포트 만들기

```
nano ~/pt_system/reports/weekly_report.py
```

```

from pathlib import Path
import sys

sys.path.append(str(Path("/home/ubuntu/pt_system")))

from google import genai
from scripts.config import GEMINI_MODEL
from scripts.db import get_conn
from reports.fetch_data import fetch_range

def build_prompt(data):
    return f"""
당신은 개인 PT 트레이너입니다.
아래의 7일 기록을 보고 한국어로 주간 리포트를 작성하세요.

포함할 내용:
1. 이번 주 요약
2. 운동 수행 평가
3. 식단 패턴 평가
4. 체중/수면/컨디션 추세
5. 다음 주 운동 방향
6. 다음 주 식단 방향
7. 주의할 점

규칙:
- 의학적 진단 금지
- 초보자가 이해하기 쉽게 작성
- 표를 1개 이상 포함
- 전체 길이 1800자 이내

데이터:
{data}
"""

def save_report(start, end, text):
    conn = get_conn()
    cur = conn.cursor()
    cur.execute(
        """
        INSERT INTO reports (report_type, start_date, end_date, report_text)
        VALUES (?, ?, ?, ?)
        """,
        ("weekly", start, end, text),
    )
    conn.commit()
    conn.close()

def main():
    data = fetch_range(days=7)
    client = genai.Client()
    response = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=build_prompt(data),
    )
    report = response.text.strip()
    save_report(data["start"], data["end"], report)
    print(report)

```

```
if __name__ == "__main__":  
    main()
```

실행합니다.

```
python3 ~/pt_system/reports/weekly_report.py
```

11장. Telegram 발송 기능 만들기

메시지 보내기 스크립트

```
nano ~/pt_system/scripts/send_telegram.py
```

```
import requests
from scripts.config import TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID

def send_message(text):
    if not TELEGRAM_BOT_TOKEN or not TELEGRAM_CHAT_ID:
        raise RuntimeError("Telegram 설정이 비어 있습니다.")

    url = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage"
    payload = {
        "chat_id": TELEGRAM_CHAT_ID,
        "text": text,
    }
    res = requests.post(url, json=payload, timeout=15)
    res.raise_for_status()
    return res.json()

if __name__ == "__main__":
    send_message("AI PT 트레이너 Telegram 발송 테스트입니다.")
    print("전송 완료")
```

테스트합니다.

```
python3 ~/pt_system/scripts/send_telegram.py
```

Telegram으로 메시지가 오면 성공입니다.

일일 리포트를 Telegram으로 보내기

```
nano ~/pt_system/reports/send_daily_report.py
```



```
from pathlib import Path
import subprocess
import sys

sys.path.append(str(Path("/home/ubuntu/pt_system")))

from scripts.send_telegram import send_message

result = subprocess.run(
    ["python3", "/home/ubuntu/pt_system/reports/daily_report.py"],
    capture_output=True,
    text=True,
    check=True,
)

send_message("[일일 PT 리포트]\n\n" + result.stdout.strip())
print("일일 리포트 전송 완료")
```

주간 리포트를 Telegram으로 보내기

```
nano ~/pt_system/reports/send_weekly_report.py
```

```
from pathlib import Path
import subprocess
import sys

sys.path.append(str(Path("/home/ubuntu/pt_system")))

from scripts.send_telegram import send_message

result = subprocess.run(
    ["python3", "/home/ubuntu/pt_system/reports/weekly_report.py"],
    capture_output=True,
    text=True,
    check=True,
)

send_message("[주간 PT 리포트]\n\n" + result.stdout.strip())
print("주간 리포트 전송 완료")
```

12장. Telegram 입력 받기

초보자용 입력 방식

Telegram 입력을 받는 방식은 두 가지가 있습니다.

방식	설명	초보자 추천
Polling	봇이 주기적으로 새 메시지를 확인	추천
Webhook	Telegram이 서버 주소로 직접 전송	나중에

이 매뉴얼은 Polling 방식으로 시작합니다.

Telegram Polling 봇 만들기

```
nano ~/pt_system/scripts/telegram_polling.py
```

```

import time
import subprocess
from pathlib import Path
import sys
import requests

sys.path.append(str(Path("/home/ubuntu/pt_system")))

from scripts.config import TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID
from scripts.send_telegram import send_message

API_URL = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}"

def get_updates(offset=None):
    params = {"timeout": 30}
    if offset:
        params["offset"] = offset
    res = requests.get(f"{API_URL}/getUpdates", params=params, timeout=40)
    res.raise_for_status()
    return res.json()["result"]

def handle_text(text):
    if text == "/daily":
        result = subprocess.run(
            ["python3", "/home/ubuntu/pt_system/reports/daily_report.py"],
            capture_output=True,
            text=True,
            check=True,
        )
        send_message("[일일 PT 리포트]\n\n" + result.stdout.strip())
        return

    if text == "/weekly":
        result = subprocess.run(
            ["python3", "/home/ubuntu/pt_system/reports/weekly_report.py"],
            capture_output=True,
            text=True,
            check=True,
        )
        send_message("[주간 PT 리포트]\n\n" + result.stdout.strip())
        return

    if text == "/help":
        send_message(
            "사용법\n"
            "- 운동/식단/체중을 자연스럽게 입력하세요.\n"
            "- /daily: 일일 리포트\n"
            "- /weekly: 주간 리포트\n"
            "- /help: 도움말"
        )
        return

    result = subprocess.run(
        ["python3", "/home/ubuntu/pt_system/scripts/save_message.py", text],
        capture_output=True,
        text=True,
        check=True,
    )

```

```

    )
    send_message(result.stdout.strip())

def main():
    if not TELEGRAM_BOT_TOKEN:
        raise RuntimeError("TELEGRAM_BOT_TOKEN이 없습니다.")

    offset = None
    send_message("AI PT 트레이너가 시작되었습니다. 오늘 기록을 보내주세요.")

    while True:
        try:
            updates = get_updates(offset)
            for update in updates:
                offset = update["update_id"] + 1
                message = update.get("message", {})
                chat_id = str(message.get("chat", {}).get("id", ""))

                if TELEGRAM_CHAT_ID and chat_id != str(TELEGRAM_CHAT_ID):
                    continue

                text = message.get("text", "").strip()
                if text:
                    handle_text(text)
        except Exception as e:
            print(f"오류: {e}")
            time.sleep(5)

if __name__ == "__main__":
    main()

```

실행합니다.

```
python3 ~/pt_system/scripts/telegram_polling.py
```

Telegram에 다음처럼 보내봅니다.

```

오늘 운동:
스쿼트 60kg 5회 5세트
벤치프레스 40kg 8회 4세트

식단:
점심 닭가슴살 200g, 밥 반 공기

체중 78.2kg, 수면 6.5시간, 컨디션 보통

```

기록 완료 메시지가 오면 성공입니다.

13장. 웹 대시보드 만들기

대시보드에서 볼 내용

대시보드는 다음을 보여줍니다.

- 오늘 운동 기록 수
- 오늘 식단 기록 수
- 최근 체중
- 최근 리포트
- 운동 기록 목록
- 식단 기록 목록
- 체중 변화

초보자에게 대시보드는 매우 중요합니다. Telegram은 입력에는 편하지만, 누적 기록을 보기에 불편하기 때문입니다.

Flask 앱 만들기

```
nano ~/pt_system/dashboard/app.py
```

```

from pathlib import Path
import sys

sys.path.append(str(Path("/home/ubuntu/pt_system")))

from flask import Flask, render_template_string
from scripts.db import get_conn

app = Flask(__name__)

HTML = """
<!doctype html>
<html lang="ko">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>AI PT 대시보드</title>
  <style>
    body { font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif; margin: 0;
background: #f6f7f9; color: #222; }
    header { background: #111827; color: white; padding: 24px; }
    main { max-width: 1100px; margin: 0 auto; padding: 24px; }
    h1 { margin: 0; font-size: 28px; }
    h2 { margin-top: 32px; font-size: 20px; }
    .grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 12px; }
    .card { background: white; border: 1px solid #e5e7eb; border-radius: 8px; padding: 16px; }
    .value { font-size: 28px; font-weight: 700; margin-top: 8px; }
    table { width: 100%; border-collapse: collapse; background: white; }
    th, td { border-bottom: 1px solid #e5e7eb; padding: 10px; text-align: left; vertical-align: top; }
    th { background: #f3f4f6; }
    pre { white-space: pre-wrap; line-height: 1.6; }
    @media (max-width: 800px) { .grid { grid-template-columns: repeat(2, 1fr); } }
    @media (max-width: 520px) { .grid { grid-template-columns: 1fr; } main { padding: 14px; } }
  </style>
</head>
<body>
  <header>
    <h1>AI PT 대시보드</h1>
    <p>운동, 식단, 체중, 리포트를 한 곳에서 확인합니다.</p>
  </header>
  <main>
    <section class="grid">
      <div class="card"><div>운동 기록</div><div class="value">{{ workout_count }}</div></div>
      <div class="card"><div>식단 기록</div><div class="value">{{ meal_count }}</div></div>
      <div class="card"><div>바이탈 기록</div><div class="value">{{ vital_count }}</div></div>
      <div class="card"><div>최근 체중</div><div class="value">{{ latest_weight or "-" }}</div></div>
    </section>

    <h2>최근 AI 리포트</h2>
    <div class="card">
      <pre>{{ latest_report or "아직 리포트가 없습니다." }}</pre>
    </div>

    <h2>최근 운동 기록</h2>
    <table>
      <tr><th>날짜</th><th>운동</th><th>메모</th></tr>
      {% for row in workouts %}

```

```

        <tr><td>{{ row.record_date }}</td><td>{{ row.exercise_name }}</td><td>{{ row.memo }}</td></tr>
        {% endfor %}
    </table>

    <h2>최근 식단 기록</h2>
    <table>
        <tr><th>날짜</th><th>구분</th><th>내용</th></tr>
        {% for row in meals %}
        <tr><td>{{ row.record_date }}</td><td>{{ row.meal_type or "-" }}</td><td>{{ row.food_text }}</td>
    </tr>
        {% endfor %}
    </table>

    <h2>최근 체중/수면</h2>
    <table>
        <tr><th>날짜</th><th>체중</th><th>수면</th><th>컨디션</th></tr>
        {% for row in vitals %}
        <tr><td>{{ row.record_date }}</td><td>{{ row.body_weight_kg or "-" }}</td><td>{{ row.sleep_hours or
    "-" }}</td><td>{{ row.condition_text or "-" }}</td></tr>
        {% endfor %}
    </table>
</main>
</body>
</html>
"""

def one_value(query):
    conn = get_conn()
    row = conn.execute(query).fetchone()
    conn.close()
    return row[0] if row else None

@app.route("/")
def index():
    conn = get_conn()

    workout_count = conn.execute("SELECT COUNT(*) FROM workouts").fetchone()[0]
    meal_count = conn.execute("SELECT COUNT(*) FROM meals").fetchone()[0]
    vital_count = conn.execute("SELECT COUNT(*) FROM vitals").fetchone()[0]

    latest_weight_row = conn.execute(
        "SELECT body_weight_kg FROM vitals WHERE body_weight_kg IS NOT NULL ORDER BY record_date DESC, id
    DESC LIMIT 1"
    ).fetchone()
    latest_weight = latest_weight_row[0] if latest_weight_row else None

    latest_report_row = conn.execute(
        "SELECT report_text FROM reports ORDER BY created_at DESC, id DESC LIMIT 1"
    ).fetchone()
    latest_report = latest_report_row[0] if latest_report_row else None

    workouts = conn.execute(
        "SELECT * FROM workouts ORDER BY record_date DESC, id DESC LIMIT 20"
    ).fetchall()
    meals = conn.execute(
        "SELECT * FROM meals ORDER BY record_date DESC, id DESC LIMIT 20"
    ).fetchall()

```

```

vitals = conn.execute(
    "SELECT * FROM vitals ORDER BY record_date DESC, id DESC LIMIT 20"
).fetchall()

conn.close()

return render_template_string(
    HTML,
    workout_count=workout_count,
    meal_count=meal_count,
    vital_count=vital_count,
    latest_weight=latest_weight,
    latest_report=latest_report,
    workouts=workouts,
    meals=meals,
    vitals=vitals,
)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

```

실행합니다.

```
python3 ~/pt_system/dashboard/app.py
```

브라우저에서 접속합니다.

```
http://서버_공인_IP:5000
```

Oracle 방화벽 확인

대시보드가 열리지 않으면 Oracle Cloud 보안 목록에서 포트 5000 이 열려 있는지 확인해야 합니다.

다만 운영 단계에서는 5000 포트를 그대로 공개하기보다 Nginx를 통해 80번 포트로 연결하는 것이 좋습니다.

초보자 실습에서는 먼저 5000 으로 확인하고, 이후 운영 단계에서 Nginx 설정을 적용합니다.

14장. systemd로 항상 실행하기

왜 systemd가 필요한가

터미널에서 직접 실행한 프로그램은 SSH 접속을 끊으면 멈출 수 있습니다.

systemd를 사용하면 서버가 재부팅되어도 자동으로 다시 실행할 수 있습니다.

이 프로젝트에서는 두 가지를 systemd로 실행합니다.

- Telegram Polling 봇
- Dashboard 웹 앱

Telegram 봇 서비스 만들기

```
sudo nano /etc/systemd/system/pt-telegram.service
```

```
[Unit]
Description=AI PT Telegram Bot
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/pt_system
ExecStart=/home/ubuntu/pt_system/venv/bin/python /home/ubuntu/pt_system/scripts/telegram_polling.py
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

서비스 등록:

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable pt-telegram
```

```
sudo systemctl start pt-telegram
```

상태 확인:

```
sudo systemctl status pt-telegram
```

Dashboard 서비스 만들기

```
sudo nano /etc/systemd/system/pt-dashboard.service
```

```
[Unit]
Description=AI PT Dashboard
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/pt_system
ExecStart=/home/ubuntu/pt_system/venv/bin/gunicorn -w 2 -b 127.0.0.1:5000 dashboard.app:app
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

서비스 등록:

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable pt-dashboard
```

```
sudo systemctl start pt-dashboard
```

상태 확인:

```
sudo systemctl status pt-dashboard
```

15장. Nginx로 대시보드 연결

Nginx가 하는 일

Flask 대시보드는 내부에서 `127.0.0.1:5000`으로 실행됩니다.

Nginx는 외부 사용자가 접속한 요청을 Flask 앱으로 전달합니다.

즉, 사용자는 다음 주소로 접속합니다.

```
http://서버_공인_IP
```

Nginx는 내부적으로 다음으로 전달합니다.

```
http://127.0.0.1:5000
```

Nginx 설정 파일 만들기

```
sudo nano /etc/nginx/sites-available/pt-dashboard
```

```
server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

활성화합니다.

```
sudo ln -s /etc/nginx/sites-available/pt-dashboard /etc/nginx/sites-enabled/pt-dashboard
```

기본 설정이 충돌하면 제거합니다.

```
sudo rm -f /etc/nginx/sites-enabled/default
```

설정 검사:

```
sudo nginx -t
```

재시작:

```
sudo systemctl restart nginx
```

브라우저에서 접속합니다.

```
http://서버_공인_IP
```

16장. 자동 리포트 예약

Cron이란 무엇인가

Cron은 정해진 시간에 명령어를 실행하는 예약 기능입니다.

이 프로젝트에서는 다음처럼 사용합니다.

- 매일 아침 8시: 기록 알림
- 매일 밤 10시 30분: 일일 리포트 전송
- 매주 일요일 밤 11시: 주간 리포트 전송

아침 알림 스크립트

```
nano ~/pt_system/scripts/morning_alert.py
```

```
from pathlib import Path
import sys

sys.path.append(str(Path("/home/ubuntu/pt_system")))

from scripts.send_telegram import send_message

send_message(
    "좋은 아침입니다.\n"
    "오늘은 운동 가능 시간, 식사 계획, 컨디션을 간단히 남겨주세요.\n"
    "예: 오늘 하체 운동 예정, 컨디션 보통, 수면 7시간"
)
```

Cron 등록

Cron 편집기를 엽니다.

```
crontab -e
```

아래 내용을 추가합니다.

```
0 8 * * * /home/ubuntu/pt_system/venv/bin/python /home/ubuntu/pt_system/scripts/morning_alert.py >>
/home/ubuntu/pt_system/logs/cron.log 2>&1
30 22 * * * /home/ubuntu/pt_system/venv/bin/python /home/ubuntu/pt_system/reports/send_daily_report.py >>
/home/ubuntu/pt_system/logs/cron.log 2>&1
0 23 * * 0 /home/ubuntu/pt_system/venv/bin/python /home/ubuntu/pt_system/reports/send_weekly_report.py >>
/home/ubuntu/pt_system/logs/cron.log 2>&1
```

등록 확인:

```
crontab -l
```

17장. 실제 사용 예시

기본 입력 예시

Telegram에 다음처럼 자연스럽게 입력합니다.

오늘 운동:
스쿼트 60kg 5회 5세트
레그프레스 100kg 10회 4세트
런닝머신 20분

식단:
아침 계란 2개, 바나나
점심 닭가슴살 200g, 밥 반 공기
저녁 연어 스텔러드

체중 78.2kg, 수면 6.5시간, 컨디션 보통

기대 응답:

[기록 완료]
- 운동: 3개
- 식단: 3개
- 바이탈: 저장됨
- 오늘의 한마디: 기록을 남긴 것 자체가 가장 중요한 시작입니다.

일일 리포트 요청

/daily

예상 리포트:

[일일 PT 리포트]

오늘은 하체 운동과 식단 기록이 모두 남아 있어 흐름이 좋습니다.
잘한 점은 운동 종목과 식사 내용을 함께 기록했다는 점입니다.
개선할 점은 단백질 섭취량을 조금 더 명확히 적는 것입니다.

내일 할 행동:

1. 아침 체중을 같은 조건에서 측정하기
2. 단백질 식품을 매 끼니 1개 이상 포함하기
3. 하체 피로가 남아 있으면 강도를 낮추기

주간 리포트 요청

/weekly

주간 리포트는 다음 내용을 포함합니다.

- 이번 주 운동 횟수
 - 식단 기록 빈도
 - 체중 변화
 - 수면 패턴
 - 다음 주 운동 방향
 - 다음 주 식단 방향
-

18장. 점검 스크립트

전체 상태 확인 파일 만들기

```
nano ~/pt_system/scripts/health_check.py
```

```

from pathlib import Path
import sys
import sqlite3
import requests

sys.path.append(str(Path("/home/ubuntu/pt_system")))

from scripts.config import DATABASE_PATH, TELEGRAM_BOT_TOKEN, GEMINI_API_KEY

def check_db():
    try:
        conn = sqlite3.connect(DATABASE_PATH)
        tables = conn.execute("SELECT name FROM sqlite_master WHERE type='table'").fetchall()
        conn.close()
        print(f"DB 정상: {len(tables)}개 테이블")
        return True
    except Exception as e:
        print(f"DB 오류: {e}")
        return False

def check_telegram():
    try:
        res = requests.get(f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/getMe", timeout=10)
        data = res.json()
        if data.get("ok"):
            print(f"Telegram 정상: @{data['result']['username']}")
            return True
        print(f"Telegram 오류: {data}")
        return False
    except Exception as e:
        print(f"Telegram 오류: {e}")
        return False

def check_env():
    ok = True
    if not GEMINI_API_KEY:
        print("GEMINI_API_KEY 없음")
        ok = False
    if not TELEGRAM_BOT_TOKEN:
        print("TELEGRAM_BOT_TOKEN 없음")
        ok = False
    if ok:
        print("환경변수 기본값 정상")
    return ok

if __name__ == "__main__":
    print("AI PT 시스템 상태 점검")
    results = [check_env(), check_db(), check_telegram()]
    print("최종 결과:", "정상" if all(results) else "확인 필요")

```

실행합니다.

```
python3 ~/pt_system/scripts/health_check.py
```

19장. 자주 나는 오류

SSH 접속이 안 될 때

확인할 것:

- IP 주소가 맞는가
- SSH 키 파일 경로가 맞는가
- 사용자명이 `ubuntu` 인가
- Oracle 인스턴스가 Running 상태인가
- 포트 22가 열려 있는가

대표 오류:

```
Permission denied
```

의미:

SSH 키가 맞지 않거나 권한 문제가 있을 가능성이 큼니다.

Telegram 메시지가 안 올 때

확인할 것:

- `.env` 의 `TELEGRAM_BOT_TOKEN` 이 맞는가
- `.env` 의 `TELEGRAM_CHAT_ID` 가 맞는가
- 봇에게 먼저 메시지를 보냈는가
- `pt-telegram` 서비스가 실행 중인가

상태 확인:

```
sudo systemctl status pt-telegram
```

로그 확인:

```
journalctl -u pt-telegram -n 50
```

Gemini 오류가 날 때

확인할 것:

- `GEMINI_API_KEY` 가 맞는가
- API 키 앞뒤에 공백이 없는가
- 모델명이 맞는가

- 무료 사용량을 초과하지 않았는가
- 서버에서 인터넷 접속이 되는가

테스트:

```
python3 ~/pt_system/scripts/test_gemini.py
```

대시보드가 안 열릴 때

확인할 것:

- `pt-dashboard` 서비스가 실행 중인가
- Nginx가 실행 중인가
- Oracle 방화벽에서 80번 포트가 열려 있는가
- Flask 앱에 오류가 없는가

상태 확인:

```
sudo systemctl status pt-dashboard
```

```
sudo systemctl status nginx
```

로그 확인:

```
journalctl -u pt-dashboard -n 50
```

DB에 기록이 안 쌓일 때

확인할 것:

- `pt_data.db` 파일이 있는가
- `init_db.py` 를 실행했는가
- `save_message.py` 테스트가 성공했는가
- 프로젝트 경로가 `/home/ubuntu/pt_system` 과 다른가

테이블 확인:

```
sqlite3 ~/pt_system/pt_data.db ".tables"
```

최근 기록 확인:

```
sqlite3 ~/pt_system/pt_data.db "SELECT * FROM raw_messages ORDER BY id DESC LIMIT 5;"
```

20장. 운영 루틴

매일 할 일

매일 사용자는 다음만 하면 됩니다.

1. 아침 체중 또는 컨디션 입력
2. 운동 후 운동 기록 입력
3. 식사 후 식단 기록 입력
4. 밤에 일일 리포트 확인

완벽하게 기록하려고 하지 않아도 됩니다. 초보자에게 가장 중요한 것은 기록을 이어가는 것입니다.

매주 할 일

매주 한 번 다음을 확인합니다.

- 운동 횟수
- 체중 변화
- 식단 기록 빈도
- 수면 부족 여부
- 너무 무리한 운동은 없었는지
- 다음 주 목표를 낮출지 높일지

주간 리포트는 일요일 밤 또는 월요일 아침에 확인하는 것을 추천합니다.

좋은 입력 습관

좋은 입력:

스쿼트 60kg 5회 5세트, 벤치프레스 40kg 8회 4세트
점심 닭가슴살 200g, 밥 반 공기, 김치
체중 78.2kg, 수면 6.5시간, 컨디션 보통

나쁜 입력:

오늘 대충 함
밥 먹음
피곤

AI는 자세한 기록이 있을수록 더 좋은 피드백을 줍니다.

21장. 백업

왜 백업이 필요한가

운동과 식단 기록은 시간이 지날수록 가치가 커집니다.

한 달치 기록보다 6개월치 기록이 훨씬 중요합니다. 서버 문제로 DB 파일을 잃어버리지 않도록 백업해야 합니다.

SQLite 백업

백업 폴더를 만듭니다.

```
mkdir -p ~/pt_system/backups
```

수동 백업:

```
cp ~/pt_system/pt_data.db ~/pt_system/backups/pt_data_$(date +%Y%m%d).db
```

백업 확인:

```
ls -lh ~/pt_system/backups
```

자동 백업 Cron

```
crontab -e
```

아래 줄을 추가합니다.

```
0 3 * * * cp /home/ubuntu/pt_system/pt_data.db /home/ubuntu/pt_system/backups/pt_data_$(date +%Y%m%d).db
```

매일 새벽 3시에 DB 파일이 백업됩니다.

22장. 보안 강화

기본 보안 체크리스트

- `.env` 권한이 `600` 인가
 - Telegram Bot Token을 공개하지 않았는가
 - Gemini API Key를 공개하지 않았는가
 - 대시보드를 공개해야 하는 이유가 있는가
 - Oracle 방화벽에서 불필요한 포트를 열지 않았는가
 - SSH 키를 안전한 곳에 보관했는가
-

대시보드 공개 범위

대시보드는 개인 건강 기록을 보여줍니다.

가능하면 다음 중 하나를 선택하세요.

1. 집 IP에서만 접속 허용
2. Cloudflare Access 같은 인증 도구 사용
3. Nginx Basic Auth 사용
4. 대시보드를 외부 공개하지 않고 SSH 터널로만 접속

초보자에게 가장 쉬운 방법은 처음에는 대시보드를 테스트용으로만 열고, 실제 운영 전에는 인증을 추가하는 것입니다.

23장. PostgreSQL로 확장하고 싶을 때

PostgreSQL이 필요한 경우

SQLite로 충분하지 않은 상황은 다음과 같습니다.

- 가족이나 회원 여러 명이 함께 사용
- 웹 서비스처럼 외부 사용자에게 제공
- 기록량이 매우 많아짐
- 권한 관리가 필요
- 외부 분석 도구와 연동

그 전까지는 SQLite를 유지하는 편이 좋습니다.

마이그레이션 개념

SQLite에서 PostgreSQL로 옮긴다는 것은 다음을 뜻합니다.

1. PostgreSQL 설치
2. DB 사용자 생성
3. 테이블 생성
4. 기존 SQLite 데이터 내보내기
5. PostgreSQL로 가져오기
6. Python 코드의 DB 연결 부분 변경

초보자에게는 이 과정이 꽤 어렵습니다. 따라서 처음부터 PostgreSQL로 시작하기보다, 시스템이 실제로 잘 쓰이는지 확인한 뒤 옮기는 것을 추천합니다.

24장. PDF 변환 전 정리

PDF에 넣으면 좋은 요소

PDF를 만들 때 다음 요소를 추가하면 읽기 편합니다.

- 표지
 - 목차
 - 각 장 시작 페이지
 - 실습 체크박스
 - 명령어 박스
 - 오류 해결 박스
 - 전체 구조 그림
 - 마지막 점검표
-

PDF 변환 전 체크리스트

- 제목이 명확한가
 - 대상 독자가 초보자로 유지되는가
 - 명령어가 한 줄씩 복사 가능하게 되어 있는가
 - API 키가 예시값으로만 들어가 있는가
 - 실제 비밀 정보가 포함되어 있지 않은가
 - SQLite를 기본값으로 설명했는가
 - PostgreSQL은 확장 선택지로 분리했는가
 - Telegram 테스트 절차가 있는가
 - Gemini 테스트 절차가 있는가
 - 대시보드 테스트 절차가 있는가
 - 오류 해결 장이 있는가
 - 보안 주의사항이 있는가
-

25장. 전체 완성 체크리스트

아래 항목을 순서대로 확인하세요.

```
[ ] Oracle VPS 생성 완료
[ ] SSH 접속 성공
[ ] Python, pip, venv 설치 완료
[ ] ~/pt_system 폴더 생성 완료
[ ] Python 가상환경 생성 완료
[ ] 필요한 Python 패키지 설치 완료
[ ] Gemini API Key 발급 완료
[ ] Telegram Bot Token 발급 완료
[ ] Telegram Chat ID 확인 완료
[ ] .env 파일 작성 완료
[ ] .env 권한 600 설정 완료
[ ] SQLite DB 생성 완료
[ ] OpenClaw workspace 생성 완료
[ ] AGENTS.md 작성 완료
[ ] SOUL.md 작성 완료
[ ] USER.md 작성 완료
[ ] Telegram 발송 테스트 성공
[ ] Gemini 테스트 성공
[ ] 메시지 저장 테스트 성공
[ ] Telegram Polling 봇 테스트 성공
[ ] 일일 리포트 생성 성공
[ ] 주간 리포트 생성 성공
[ ] Dashboard 접속 성공
[ ] systemd 서비스 등록 완료
[ ] Cron 자동 리포트 등록 완료
[ ] 백업 설정 완료
[ ] 보안 점검 완료
```

부록 A. 명령어 모음

서비스 상태 확인

```
sudo systemctl status pt-telegram
```

```
sudo systemctl status pt-dashboard
```

```
sudo systemctl status nginx
```

서비스 재시작

```
sudo systemctl restart pt-telegram
```

```
sudo systemctl restart pt-dashboard
```

```
sudo systemctl restart nginx
```

로그 확인

```
journalctl -u pt-telegram -n 50
```

```
journalctl -u pt-dashboard -n 50
```

```
tail -n 50 ~/pt_system/logs/cron.log
```

DB 확인

```
sqlite3 ~/pt_system/pt_data.db ".tables"
```

```
sqlite3 ~/pt_system/pt_data.db "SELECT * FROM reports ORDER BY id DESC LIMIT 1;"
```

부록 B. PT 트레이너용 프롬프트 예시

일일 피드백 프롬프트

너는 개인 PT 트레이너다.

오늘의 운동, 식단, 체중, 수면 기록을 보고 짧은 피드백을 작성해라.

규칙:

- 한국어
- 800자 이내
- 잘한 점 2개
- 개선점 2개
- 내일 행동 3개
- 의학적 진단 금지
- 통증이 있으면 전문가 상담 권장

주간 피드백 프롬프트

너는 개인 PT 트레이너다.

최근 7일 기록을 보고 주간 리포트를 작성해라.

포함할 내용:

- 이번 주 요약
- 운동 진행 상황
- 식단 패턴
- 체중 변화
- 수면과 컨디션
- 다음 주 운동 계획
- 다음 주 식단 계획

주의:

- 초보자가 이해할 수 있게 설명
- 무리한 감량 권장 금지
- 데이터가 부족하면 부족하다고 말하기

부록 C. 참고 자료

이 매뉴얼은 다음 공식 문서를 참고해 작성 방향을 잡았습니다.

- Google AI for Developers, Gemini API Quickstart: <https://ai.google.dev/gemini-api/docs/quickstart>
- Google AI for Developers, Gemini model documentation: <https://ai.google.dev/gemini-api/docs/models/gemini>
- OpenClaw Docs, Agent runtime: <https://docs.openclaw.ai/agent>
- OpenClaw Docs, Default AGENTS.md: <https://docs.openclaw.ai/reference/AGENTS.default>

공식 문서는 수시로 바뀔 수 있습니다. 특히 API 모델명, 설치 명령어, 요금 정책은 PDF 배포 전에 다시 확인하는 것이 좋습니다.

마무리

이 매뉴얼의 목표는 완벽한 헬스케어 서비스를 만드는 것이 아닙니다.

목표는 내가 매일 쉽게 기록하고, AI가 그 기록을 바탕으로 꾸준히 피드백해주는 개인 PT 시스템을 갖추는 것입니다.

처음에는 기능이 단순해도 괜찮습니다.

중요한 순서는 다음과 같습니다.

1. Telegram으로 기록이 들어온다.
2. DB에 저장된다.
3. Gemini가 리포트를 만든다.
4. 대시보드에서 확인된다.
5. 매일 자동으로 반복된다.

이 다섯 가지가 되면, 이미 개인 AI PT 트레이너의 기본 구조는 완성된 것입니다.